Licence 3 Informatique - Informatique & Applications
UFR Mathématique et Informatique
Université Paris Cité.

Rapport TPs
Algorithmique Avancée

Prénom et Nom : **Melissa YAHIATENE**

TP0 :

Partie 1 : (TP11 Programmation avancée)

Le but de ce programme est de représenter des formes polygonales composées d'une suite de points repérés dans l'espace grâce à leurs coordonnées x et y. Et ce, en utilisant des listes doublement chainées.

Procédure :

- ✧ J'ai défini une structure pour représenter un ***point*** dans le plan caractérisé par ses coordonnées x et y :

```
typedef struct point{
    int x, y;
}point;
```

- ✧ J'ai ensuite défini la structure ***cellule*** qui représente un élément de la liste doublement chaînée contenant un point dans le plan. Elle permet une navigation dans les deux directions (avant et arrière) et une manipulation dynamique des éléments.

```
typedef struct cellule{
    point p;
    struct cellule *pred, *succ;
}cellule;
```

- ✧ Le programme permet :
  - de créer un point et une cellule correspondante à ce point
  - d'insérer une cellule à une position donnée (dans la liste des cellules représentant la forme polygonale)
  - de supprimer une cellule à une position donnée (de la liste des cellules)
  - d'afficher la liste des points représentant la forme polygonale

- ✧ Fichiers :
  - liste.h : définition des structures et des prototypes.
  - liste.c : implémentation des fonctions (NouvelCel, InsererCellule, SupprimerCellule, afficher).
  - maListe.c : programme principal qui utilise la liste.

Utilisation **:**

Au lancement, le programme demande à l'utilisateur :

- d'entrer les coordonnées des points (x, y),
- de choisir l'emplacement d'insertion,
- de décider s'il veut ajouter d'autres points,
- puis d'afficher la liste.
- ensuite, l'utilisateur peut supprimer des cellules en indiquant leur position.

Exemple :

```
Insertion des points :
Inserer les coordonnees X et Y du Point(1) dans cet ordre: 1 2
A quel emplacement voulez-vous inserer cette cellule ? 1

Voulez vous inserer un autre point ? 0: NON | 1: OUI 1
Inserer les coordonnees X et Y du Point(2) dans cet ordre: 3 4
A quel emplacement voulez-vous inserer cette cellule ? 2

Voulez vous inserer un autre point ? 0: NON | 1: OUI 1
Inserer les coordonnees X et Y du Point(3) dans cet ordre: 5 6
A quel emplacement voulez-vous inserer cette cellule ? 3

Voulez vous inserer un autre point ? 0: NON | 1: OUI 1
Inserer les coordonnees X et Y du Point(4) dans cet ordre: 7 8
A quel emplacement voulez-vous inserer cette cellule ? 4

Voulez vous inserer un autre point ? 0: NON | 1: OUI 0
(1,2)
(3,4)
(5,6)
(7,8)

Quel est l'emplacement de la cellule que vous souhaitez supprimer ? 4

Voulez vous supprimer un autre point ? 0: NON | 1: OUI 1

Quel est l'emplacement de la cellule que vous souhaitez supprimer ? 1

Voulez vous supprimer un autre point ? 0: NON | 1: OUI 0
(3,4)
(5,6)
```

<u>Partie 2</u> : (TP12 Programmation avancée)

Le but de ce programme est de représenter un arbre binaire avec une représentation chaînée où chaque nœud de l'arbre stockera un caractère (fourni par l'utilisateur), son numéro de création (calculé automatiquement), l'adresse du nœud représentant son fils gauche et l'adresse du nœud représentant son fils droit.

<u>Procédure</u> **:**

- ✧ J'ai défini une structure pour représenter un ***nœud*** caractérisé par un caractère(val), un numéro de création(num), un pointeur vers son fils gauche et un pointeur vers son fils droit :

```
typedef struct noeud{
    char val;
    int num;
    struct noeud *fils_gauche;
    struct noeud *fils_droit;
}noeud;
```

- ✧ J'ai ensuite défini les différentes fonctions pour manipuler les nœuds de l'arbre :
  - Création d'un noeud avec un caractère et attribution automatique d'un numéro.
  - Insertion d'un noeud comme fils gauche ou fils droit d'un noeud existant.
  - Recherche d'un noeud par son numéro.
  - Parcours préfixe (racine → gauche → droite).
  - Gestion d'un tableau des numéros pour vérifier l'existence des noeuds.

```
//creer un noeud et l'initialiser
noeud *nouvNoeud(char carac);

//rechercher un noeud dans un arbre
noeud *rechercheNoeud(noeud *n, int num_noeud);

//Insertion d'un noeud comme fils gauche
void insererFG(noeud *newNoeud, noeud *arbre, int num_pere);

//Insertion d'un noeud comme fils droit
void insererFD(noeud *newNoeud, noeud *arbre, int num_pere);

//Parcours Prefixe d'un arbre
void parcoursPrefixe(noeud *racine);

//fonctions utilitaires :
//sauvegarder les numeros des noeuds créés
void ajoutNumero(int n, int tab[n], int num);

//verifier si un numero a deja été attribué
int existeNum(int n, int tab[n], int num);
```

✧ Fichiers :

- operations.h : définition de la structure `noeud` et des prototypes.
- operations.c : implémentation des fonctions (création, insertion, recherche, parcours, utilitaires).
- mainArbre.c : programme principal qui permet de créer un arbre et de tester les fonctions.

<u>Utilisation</u> :

1. Le programme demande le caractère de la racine.
2. Ensuite, l'utilisateur peut insérer des noeuds :

   - saisir le caractère du nouveau noeud,

   - indiquer le numéro du père (doit exister),

   - choisir gauche ou droite.
3. Le programme affiche le parcours préfixe de l'arbre construit.
4. L'utilisateur peut rechercher un noeud par son numéro et voir son caractère.

Exemple :

```
=============================================
Creation d'un arbre :
Caractere de la racine ? B

Insersion de noeuds :
Caractere du nouveau noeud a inserer ? A
Numero de son pere ? (inserer un numero d'un noeud existant !) : 1
Inserer A gauche OU A droite ? (1:GAUCHE  |  2:DROITE) : 1

Voulez-vous inserer un autre noeud ? ( 1:OUI | 2:NON) : 1
Caractere du nouveau noeud a inserer ? O
Numero de son pere ? (inserer un numero d'un noeud existant !) : 1
Inserer A gauche OU A droite ? (1:GAUCHE  |  2:DROITE) : 2

Voulez-vous inserer un autre noeud ? ( 1:OUI | 2:NON) : 1
Caractere du nouveau noeud a inserer ? R
Numero de son pere ? (inserer un numero d'un noeud existant !) : 1
Inserer A gauche OU A droite ? (1:GAUCHE  |  2:DROITE) : 1

Voulez-vous inserer un autre noeud ? ( 1:OUI | 2:NON) : 1
Caractere du nouveau noeud a inserer ? V
Numero de son pere ? (inserer un numero d'un noeud existant !) : 4
Inserer A gauche OU A droite ? (1:GAUCHE  |  2:DROITE) : 2

Voulez-vous inserer un autre noeud ? ( 1:OUI | 2:NON) : 2

=============================================
Parcours prefixe de l'arbre :
B       R       A       V       O
=============================================
recherche d'un noeuds :
Quel est le numero du noeud que vous recherchez ? 4
< R >
```

TP1 :

Le but de ce programme est d'implémenter en C l'algorithme de coloration de Welsh-Powell en utilisant la structure de donnée dite de matrice d'adjacence (int **adjacence) pour représenter des graphes.

Procédure :

- ✧ J'ai défini une structure de donnée ***graph*** pour représenter un graphe par son nombre de sommets et sa matrice d'adjacence :

```
typedef struct graph{
    int nbSommets;
    int **adjacence;
}graph;
```

- ✧ J'ai également défini une structure ***sommet*** pour représenter un sommet avec son numéro ainsi que son degré (le nombre de ses voisins) :

```
typedef struct sommet{
    int num;
    int degre;
}sommet;
```

- ✧ J'ai ensuite implémenté différentes fonctions pour créer un graphe ajouter des arêtes, et des fonctions utilitaires à utiliser dans la fonction welshPowell qui permet d'appliquer l'algorithme Welsh Powell de coloration de graphe, et d'afficher le résultat :

  - Création d'un graphe avec un nombre de sommets donné.
  - Ajout interactif d'arêtes entre sommets.
  - Calcul du degré de chaque sommet.
  - Tri des sommets par degré décroissant.
  - Coloration du graphe avec l'algorithme de Welsh-Powell.
  - Affichage du nombre de couleurs utilisées et de la couleur attribuée à chaque sommet.
  - Libération de la mémoire allouée.

```
//creer un graphe caracterisé par son nombre de sommets et sa matrice d'adjacence
graph *creerGraph(int n);

//ajouter une arete entre deux somment donnés
void ajouterArete(graph *g, int s1, int s2);
```

```
//calculer les degrés de chaque sommet
sommet* calculerDegre(graph *g);


//fonction utilitaire pour "qsort" (pour comparer deux elements)
int compareDesc(const void *a, const void *b);

//tri decroissant des degrés
sommet* trierDegre(graph *g);

//implementation de l'algorithme Welsh Powell pour la coloration de graphe
void welshPowell(graph *g);

//liberer l'espace alloué pour le graphe
void libererGraph(graph *g);
```

- Fichiers :

  - coloration.h : définitions des structures et prototypes des fonctions.

  - coloration.c : implémentation des fonctions (création, ajout d'arêtes, calcul des degrés, tri, coloration, libération).

  - mainGraph.c : programme principal qui permet de créer un graphe interactif et de lancer la coloration.

Utilisation :

1. Le programme demande le nombre de sommets du graphe.

2. L'utilisateur ajoute les arêtes en saisissant les indices des sommets à relier.

   - Les indices doivent être compris entre 0 et n-1.

3. Le programme applique l'algorithme de Welsh-Powell et affiche :

   - le nombre de couleurs utilisées,

   - la couleur attribuée à chaque sommet.

Exemple :

```
Ajouter les aretes :
Les sommets a relier avec une arete ? (s1 s2) : 0 3

Voulez-vous ajouter une autre arete ? (1:OUI | 2:NON) : 1
Les sommets a relier avec une arete ? (s1 s2) : 0 6

Voulez-vous ajouter une autre arete ? (1:OUI | 2:NON) : 1
Les sommets a relier avec une arete ? (s1 s2) : 1 2

Voulez-vous ajouter une autre arete ? (1:OUI | 2:NON) : 1
Les sommets a relier avec une arete ? (s1 s2) : 2 3

Voulez-vous ajouter une autre arete ? (1:OUI | 2:NON) : 1
Les sommets a relier avec une arete ? (s1 s2) : 2 4

Voulez-vous ajouter une autre arete ? (1:OUI | 2:NON) : 1
Les sommets a relier avec une arete ? (s1 s2) : 3 6

Voulez-vous ajouter une autre arete ? (1:OUI | 2:NON) : 1
Les sommets a relier avec une arete ? (s1 s2) : 4 5

Voulez-vous ajouter une autre arete ? (1:OUI | 2:NON) : 1
Les sommets a relier avec une arete ? (s1 s2) : 6 7

Voulez-vous ajouter une autre arete ? (1:OUI | 2:NON) : 2

==================================================
Coloration du graphe:

Ordre de marquage :
1er  sommet colore: 6  Couleur: 0
1eme sommet colore: 2  Couleur: 0
2eme sommet colore: 3  Couleur: 1
3eme sommet colore: 4  Couleur: 1
4eme sommet colore: 0  Couleur: 2
5eme sommet colore: 5  Couleur: 2
6eme sommet colore: 7  Couleur: 2
7eme sommet colore: 1  Couleur: 2

Le graphe a ete colore avec <3> couleurs
Sommet 0: Couleur: 2
Sommet 1: Couleur: 2
Sommet 2: Couleur: 0
Sommet 3: Couleur: 1
Sommet 4: Couleur: 1
Sommet 5: Couleur: 2
Sommet 6: Couleur: 0
Sommet 7: Couleur: 2
```

TP2 :

Partie 1 : (Suite TP0/Partie2)

Le but de ce programme est d'implémenter les parcours préfixes, infixes et post-fixes sur les arbres binaires en reprenant le **TP0/Partie2** et les tester sur un arbre créé.

Procédure :

- ✧ J'ai rajouté au programme du TP0 sur les arbres des fonctions de parcours infixe et suffixe :

```
//Parcours Infixe d'un arbre
void parcoursInfixe(noeud *racine);

//Parcours Suffixe d'un arbre
void parcoursSuffixe(noeud *racine);
```

Utilisation :

Le programme crée automatiquement un arbre avec les noeuds suivants :
- Racine : 'B'
- Fils gauche : 'A'
- Fils droit : 'O'
- Autres noeuds insérés : 'R', 'V'

```
noeud *racine = nouvNoeud('B');
    noeud *n1 = nouvNoeud('A');
    noeud *n2 = nouvNoeud('O');
    noeud *n3 = nouvNoeud('R');
    noeud *n4 = nouvNoeud('V');

    insererFG(n1, racine, 1);
    insererFD(n2, racine, 1);
    insererFG(n3, racine, 1);
    insererFD(n4, racine, 4);
```

Ensuite, il affiche :
- Le parcours préfixe,
- Le parcours infixe,
- Le parcours suffixe.

```
==================================================
Parcours prefixe de l'arbre :
B       R       A       V       O
==================================================
Parcours infixe de l'arbre :
A       R       V       B       O
==================================================
Parcours suffixe de l'arbre :
A       V       R       O       B
```

Partie 2 :

Le but de ce programme est de tester le programme de coloration de graphe Welsh Powell du **TP1.**

Procédure :

- ✧ J'ai ajouté la fonction **Chargement d'un graphe :**
  - soit par saisie clavier,
  - soit depuis un fichier texte.

  (J'ai également rajouté une fonction utilitaire pour récupérer les clés des sommets depuis un fichier dans le cas où les sommets d'un graphe possèdent des clés comme dans le cas du graphe des pays)

1) Chargement d'un graphe par saisie clavier :

```
graph *chargerGraph(){
    int n;
    int s1, s2, rep;
    graph *g;

    printf("Nombre de sommets du graphe ? : ");
    fflush(stdout);
    scanf("%d", &n);

    g = creerGraph(n);

    printf("Ajouter les aretes :\n");
    fflush(stdout);

    do{

        do{
            printf("Les sommets a relier avec une arete ? (s1 s2) : ");
            fflush(stdout);
            scanf("%d %d", &s1, &s2);

        }while(s1<0 || s1>=n || s2<0 || s2>=n);

        ajouterArete(g, s1, s2);

        printf("\nVoulez-vous ajouter une autre arete ? (1:OUI | 2:NON) : ");
        fflush(stdout);
        scanf("%d", &rep);

    }while(rep==1);

    return g;
}
```

2) Chargement d'un graphe depuis un fichier texte :

```
graph *chargerGraphFich(const char *nomFichier, char ***cles){
    int n;
    graph *g;
    *cles = chargerCles(nomFichier);

    FILE *f = NULL;
    if(nomFichier != NULL)
    {
        f = fopen(nomFichier, "r");

        // Si le fichier n'existe pas
        if(!f){
            printf("Fichier introuvable !\n");
        }
        else{
            char ligne[1000];
            //Lire et ignorer la première ligne (les clés)
            fgets(ligne, sizeof(ligne), f);

            //Lire le nombre de sommets
            fscanf(f, "%d", &n);
            g = creerGraph(n);

            //Remplissage de la matrice
            for (int i = 0; i < (g->nbSommets); i++){
                for(int j = 0; j< (g->nbSommets); j++)
                    fscanf(f, "%d", &(g->adjacence[i][j]));
            }

            fclose(f);
        }
    }
    return g;
}
```

- La fonction utilitaire pour charger les clés : (Se trouvant sur la première ligne du fichier)

```
char** chargerCles(const char *nomFichier){
    FILE *f = fopen(nomFichier, "r");
    if (!f) {
        printf("Erreur ouverture fichier\n");
        exit(1);
    }

    //Lecture de la première ligne : les cles
    char ligne[1000];
    fgets(ligne, sizeof(ligne), f);

    //Compter les noms
    int count = 0;
    char *token = strtok(ligne, " \n");
    char **cles = NULL;
```

```c
    while(token != NULL) {
        cles = realloc(cles, (count + 1) * sizeof(char*));
        cles[count] = malloc(strlen(token) + 1);
        strcpy(cles[count], token);
        count++;
        token = strtok(NULL, " \n");
    }
    return cles;
}
```

- Exemple du fichier Carte.txt :

```
FRANCE ESPAGNE PORTUGAL ANDORRE ITALIE AUTRICHE SUISSE ALLEMAGNE LUXEMBOURG BELGIQUE PAYS-BAS
11
0 1 0 1 1 0 1 1 1 1 0
1 0 1 1 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0 0 0 0
1 1 0 0 0 0 0 0 0 0 0
1 0 0 0 0 1 1 0 0 0 0
0 0 0 0 1 0 1 1 0 0 0
1 0 0 0 1 1 0 1 0 0 0
1 0 0 0 0 1 1 0 1 1 1
1 0 0 0 0 0 0 1 0 1 0
1 0 0 0 0 0 0 1 1 0 1
0 0 0 0 0 0 0 1 0 1 0
```

✧ Fichiers :
- coloration.h : définitions des structures et prototypes des fonctions.
- coloration.c : implémentation des fonctions (création, chargement calcul des degrés, tri, coloration, libération).
- mainGraph.c : programme principal qui charge un graphe et lance la coloration.
- Carte.txt : fichier exemple contenant les clés (noms des sommets), le nombre de sommets et la matrice d'adjacence.

✧ Le main :

```c
int main(){

    char **cles;
    graph *g = chargerGraphFich("Carte.txt", &cles);

    printf("\n=====================================================\n");
    fflush(stdout);

    printf("Coloration du graphe:\n");
    fflush(stdout);

    welshPowell(g, cles);

    libererCles(cles, g->nbSommets);
    libererGraph(g);

}
```

Exemple d'exécution :

```
==================================================
Coloration du graphe:

Ordre de marquage :
1er  sommet colore: FRANCE      Couleur: 0
2eme sommet colore: ALLEMAGNE   Couleur: 1
3eme sommet colore: SUISSE      Couleur: 2
4eme sommet colore: BELGIQUE    Couleur: 2
5eme sommet colore: AUTRICHE    Couleur: 2
6eme sommet colore: ITALIE      Couleur: 2
7eme sommet colore: LUXEMBOURG  Couleur: 3
8eme sommet colore: ESPAGNE     Couleur: 3
9eme sommet colore: PAYS-BAS    Couleur: 3
10eme sommet colore: ANDORRE    Couleur: 3
11eme sommet colore: PORTUGAL   Couleur: 3

Le graphe a ete colore avec <4> couleurs
Sommet: FRANCE       Couleur: 0
Sommet: ESPAGNE      Couleur: 1
Sommet: PORTUGAL     Couleur: 0
Sommet: ANDORRE      Couleur: 2
Sommet: ITALIE       Couleur: 1
Sommet: AUTRICHE     Couleur: 0
Sommet: SUISSE       Couleur: 2
Sommet: ALLEMAGNE    Couleur: 1
Sommet: LUXEMBOURG   Couleur: 3
Sommet: BELGIQUE     Couleur: 2
Sommet: PAYS-BAS     Couleur: 0
```

TP3 :

Partie 1 :

Le but de ce programme est de transformer la structure de données du TP0 pour les arbres binaires en liste chaînée pour représenter un arbre quelconque (pas seulement binaire) ou un graphe comme celui de l'exercice sur les pays européens du TD0.

Procédure :

- J'ai défini une structure ***voisin*** pour représenter le voisin d'un sommet. Il est caractérisé par son numéro et un pointeur vers le prochain voisin si le sommet possède plusieurs voisins :

```
typedef struct voisin{
    int num;
    struct voisin *suiv;
}voisin;
```

- J'ai ensuite défini une structure ***sommet*** qui représente un sommet d'un graphe, caractérisé par un caractère(val) son numéro(num : son numéro de création dans ce cas), ainsi qu'une liste de ses voisins :

```
typedef struct sommet{
    char val;
    int num;
    voisin *listeVoisins; //la liste des voisins d'un sommet
}sommet;
```

- Enfin, une structure ***graph*** représentant un graphe caractérisé par son nombre de sommets et la liste de ses sommets :

```
typedef struct graph{
    int nbSommets; //le nombre de sommets
    sommet *sommets; //liste des sommets du graphes
}graph;
```

- J'ai modifié la fonction nouvNoeud des arbres pour avoir une fonction nouvSommet qui crée et initialise un sommet du graphe :

```
sommet *nouvSommet(char carac){
    static int compteur=0;
    compteur++;

    sommet *s = (sommet*)malloc(sizeof(sommet));
    if(n==NULL){
        printf("Probleme d'allocation memeoire !");
        exit(1);
    }

    (s->val)=carac;
    (s->num)=compteur;
    (s->listeVoisins) = NULL;

    return s;
}
```

- J'ai également ajouté une fonction pour créer un graphe d'un nombre donné de sommets :

```
graph *creerGraph(int n){
    graph *g = (graph*)malloc(sizeof(graph));
    if(g==NULL){
        printf("Probleme d'allocation memeoire !");
        exit(1);
    }

    g->nbSommets=n;

    g->sommets = (sommet*)malloc(n*sizeof(sommet));

    return g;
}
```

- J'ai aussi ajouté une fonction qui permet de créer une arête entre de sommets :

```
void ajouterArete(graph *g, int s1, int s2){

    if(s1>(g->nbSommets) || s1<1){
        printf("Le sommet %d n'appartient pas au graphe !\n", s1);
    }
    else if(s2>(g->nbSommets) || s2<1){
        printf("Le sommet %d n'appartient pas au graphe !\n", s2);
    }
    else{
        // Ajouter s2 dans la liste des voisins de s1
        voisin *v1 = (voisin*)malloc(sizeof(voisin));
        v1->num = s2;
        v1->suiv  = g->sommets[s1].listeVoisins;
        g->sommets[s1].listeVoisins = v1;

        // Ajouter s1 dans la liste des voisins de s2
        voisin *v2 = (voisin*)malloc(sizeof(voisin));
        v2->num = s1;
        v2->suiv  = g->sommets[s2].listeVoisins;
        g->sommets[s2].listeVoisins = v2;
    }
}
```

- Et enfin, une fonction pour afficher le graphe, et une fonction pour libérer les espaces mémoire alloués :

```
// Afficher le graphe
void afficherGraph(graph *g) {
    for (int i = 1; i <=g->nbSommets; i++) {
        printf("Sommet %c (%d) : ", g->sommets[i].val, g->sommets[i].num);
        voisin *v = g->sommets[i].listeVoisins;
        while (v != NULL) {
            printf("%c (%d) ", g->sommets[v->num].val, v->num);
            v = v->suiv;
        }
        printf("\n");
    }
}
```

```c
// Libérer la mémoire
void libererGraph(graph *g) {
    for (int i = 1; i <= g->nbSommets; i++) {
        voisin *v = g->sommets[i].listeVoisins;
        while (v != NULL) {
            voisin *tmp = v;
            v = v->suiv;
            free(tmp);
        }
    }
    free(g->sommets);
    free(g);
}
```

Exemple de test :
Le main :

```c
int main(){
    int n = 4;
    graph *g = creerGraph(n);

    // Création des sommets
    g->sommets[1] = *nouvSommet('A');
    g->sommets[2] = *nouvSommet('B');
    g->sommets[3] = *nouvSommet('C');
    g->sommets[4] = *nouvSommet('D');

    // Ajouter quelques arêtes
    ajouterArete(g, 1, 2); // A-B
    ajouterArete(g, 1, 3); // A-C
    ajouterArete(g, 2, 3); // B-C
    ajouterArete(g, 3, 4); // C-D

    // Afficher le graphe
    printf("Affichage du graphe :\n");
    afficherGraph(g);

    // Libérer la mémoire
    libererGraph(g);
}
```

L'exécution :

```
Affichage du graphe :
Sommet A (1) : C (3) B (2)
Sommet B (2) : C (3) A (1)
Sommet C (3) : D (4) B (2) A (1)
Sommet D (4) : C (3)
```

Partie 2 :

Le but de ce programme est de corriger le programme de Welsh Powell de ChatGPT :

Procédure **:**

Les erreurs rectifiées :

1) color[MAX_VERTICES] était mal initialisé :
   Erreur :

```
int color[MAX_VERTICES] = {-1};
```

   Rectification :

```
for (i = 0; i < MAX_VERTICES; i++) {

   color[i] = -1;

}
```

2) Ajout d'un contrôle sur les indices des sommets à relier : (doivent être compris entre 0 et n-1) :

```
   if(u < 0 || u >= graph->vertices || v < 0 || v >= graph->vertices){
      printf("Erreur: Indices sommets hors bornes !\n");
   }
```

3) L'assignation des couleurs :
   Erreur : Dans le code donné, pendant l'assignation des couleurs, on teste uniquement la couleur courante, et on crée une nouvelle couleur à chaque fois que la couleur courante est déjà attribuée à un voisin, or il faut vérifier parmi toutes les couleurs créées et voir s'il existe une couleur non attribuée pour les voisins du sommet courant. Et dans le cas contraire, on crée une nouvelle couleur :

```
for (int i = 1; i < graph->vertices; i++) {
   int vertex = sorted_vertices[i];
   if (color[vertex] == -1) {
      int uncolored_adjacent = 1;
      for (int j = 0; j < graph->vertices; j++) {
         if (graph->adjMatrix[vertex][j] == 1 && color[j] == current_color) {
            uncolored_adjacent = 0;
            break;
         }
      }

      // Assign a different color to the uncolored adjacent vertices
      if (uncolored_adjacent) {
         color[vertex] = current_color;
      } else {
         current_color++;
         color[vertex] = current_color;
```

```
        }
    }
}
```

Rectification :

```
//on teste toutes les couleurs
        for(int c=0; c<=current_color && couleurAttribuee==-1; c++){
            int uncolored_adjacent = 1;
            for (j = 0; j < graph->vertices; j++) {
                if (graph->adjMatrix[vertex][j] == 1 && color[j] == c) {
                    uncolored_adjacent = 0;
                    break;
                }
            }
            if(uncolored_adjacent){
                color[vertex]=c;
                couleurAttribuee=c;
            }
        }
        if(couleurAttribuee==-1){
            color[vertex] = ++current_color; //on lui attribue une nouvelle couleur
        }
```

Exécution du code corrigé :

```
Graph colored using 3 colors.
Vertex 0: Color 0
Vertex 1: Color 1
Vertex 2: Color 2
Vertex 3: Color 1
Vertex 4: Color 0
```